



FIREBRAND

LYCEUM EDUCATION GROUP

Functions and Code Modularity

Python Pathway | Session 5

Session Recording & Professional Expectations

Session Recording

- All Estio training sessions are recorded to:
 - Support learner progress
 - Enable review of content
 - Maintain quality and compliance standards
- By attending this session, you acknowledge that recording is taking place.

Camera & Participation Expectations

- To support engagement and attendance monitoring:
 - Cameras should remain on during the session
 - Please participate actively in discussions and activities
 - If you are unable to use your camera or microphone for any reason, please inform your tutor directly via chat or message.

This session forms part of your professional apprenticeship experience. Please engage respectfully and professionally throughout.

Safeguarding, British Values & Wider Learning

Safeguarding protects individuals from harm, abuse, neglect, radicalisation and exploitation.

If you have a concern about yourself or someone else, email:

safeguarding@bpp.com

Or speak to your tutor privately.

We treat all concerns seriously and confidentially.

Safeguarding



The British Values are:

1. **Democracy**
2. **Rule of Law**
3. **Individual Liberty**
4. **Mutual Respect**
5. **Tolerance of Different Faiths and Beliefs**

These values underpin everything we do in professional and digital environments.

British Values



Prevent is about protecting people from being drawn into extremism or radicalisation.

In our programmes, Prevent links to:

- **Online safety**
- **Digital responsibility**
- **Critical thinking**
- **Professional ethics**

Prevent



As we go through today's session, consider:

How today's topic connects to British Values

Whether there are any Prevent or digital safety themes

How this learning supports your professional and career development

Your tutor will guide discussion and help you make these connections.

Competencies and Coverage

Professional Discussion underpinned by Portfolio

Knowledge

K1, K3, K4, K5, **K7**, K8,
K10, K12

Skills

S2, S3, S5, S8, S9, S13,
S14, S15, S17

Behaviours

B1, B4, B5, B6, B7, B8,
B9

Work-based Project with Questioning

Knowledge

K2, K6, K9, K11

Skills

S1, S4, S6, **S7**, S10,
S11, S12, S16

Behaviours

B2, B3

Lesson Aim and Objectives

Lesson Aim: Functions and Code Modularity

Lesson Objectives

Apply

- Apply functions to modularise and organise code

Analyse

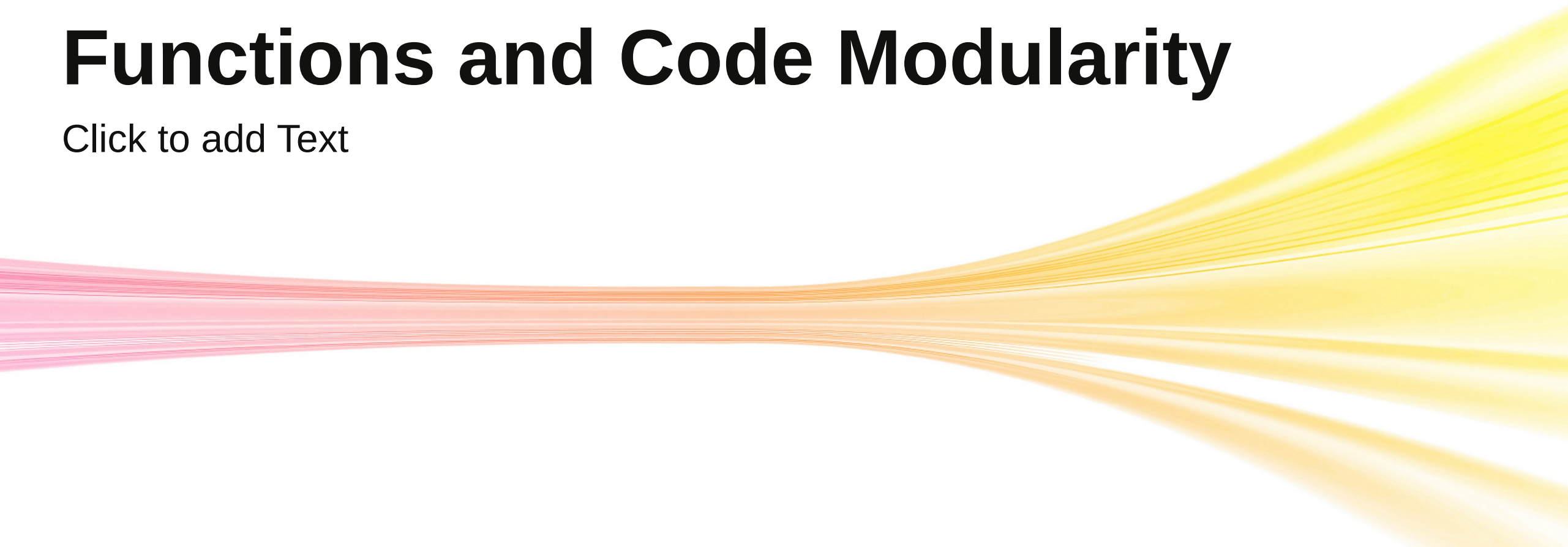
- Analyse code to identify duplication and inefficiencies

Evaluate

- Create reusable components aligned to design principles

Functions and Code Modularity

Click to add Text

A decorative graphic consisting of several overlapping, wavy lines that originate from the left side of the slide and extend towards the right. The lines are colored in a gradient from light pink on the left, through orange, to bright yellow on the right. They have a soft, ethereal quality, resembling smoke or light trails.

Introduction to Functions

Demystifying Modular Code Blocks

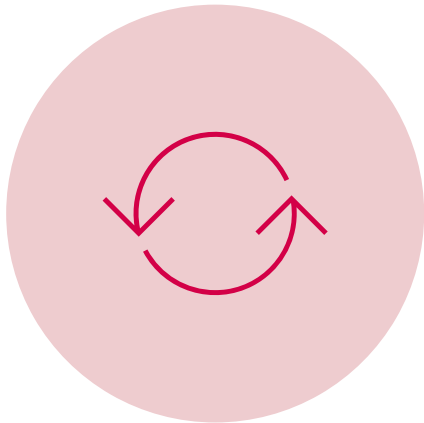
Defining a basic reusable function

```
def greet_user():  
    print("Welcome to the system!")
```

Invoking the function

```
greet_user()
```

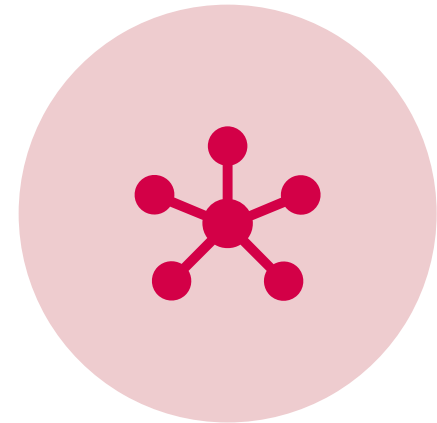
Core Concepts



REUSABLE BLOCKS



WRITE ONCE



**MODULAR
FOUNDATION**

Passing Data In and Out

Inputs (Parameters) and Outputs (Return Values)

```
def calculate_tax(amount):  
    tax_total = amount * 0.20  
    return tax_total
```

Passing data between blocks

```
final_tax = calculate_tax(100.0)
```

The Data Flow:

- **Parameters:** Variables that accept incoming data inputs
- **Arguments:** The actual data values passed during invocation
- **Return statement:** Sends computed data back to main execution

Monolithic vs. Modular Code

The Monolithic Mess (Anti-Pattern):

Everything is squeezed into one unreadable script

```
price = 200.0
discount = price * 0.10
tax = (price - discount) * 0.20
final_total = price - discount + tax
print(f'Total: {final_total}')
```

The Modular Solution (Best Practice):

Segregated, readable, and highly maintainable components

```
def apply_discount(p): return p * 0.90
def add_tax(p): return p * 1.20

final_total = add_tax(apply_discount(200.0))
```

Why Component Isolation Wins

Readability Boost:

- Functions act as self-documenting semantic code labels
- Abstract complex operations away from the main loop

Code Reuse:

- Execute identical calculations across multiple program modules
- Update bugs once to fix them universally

Team Scalability:

- Multiple developers can build separate functions simultaneously
- Isolating errors simplifies functional unit testing patterns



FIREBRAND

LYCEUM EDUCATION GROUP

Refactoring Workshop

The Flawed Starter Script

- Open the file **Refactor_Activity.py**
- The script contains messy variable names, copy-pasted calculation blocks, and zero structural organisation

Workshop Mission

- **Objective:** Transform a chaotic, copy-pasted monolithic script into a clean architectural layout
- **Core Rule:** Reduce repeated code blocks down to single, modular definitions.
- **Safety Target:** Ensure the system's external behaviour remains completely unchanged

Step-by-Step Refactoring Strategy

1. Spot and Isolate

- Scan the script lines to identify identical logic blocks repeated for each item

2. Extract & Generalize

- Cut out the duplicated math and paste it into isolated, parameter-driven def functions

3. Semantic Renaming

- Replace ambiguous names like p1, d1, and t1 with clear, descriptive terms

4. Verify Behaviour

- Run your new modular version to confirm the output match totals from line one

Expense Calculator System



ACTIVITY



Objective: Separate input, calculation and output into distinct functions

Tasks / Requirements:

- Write a function to collect an expense amount and category
- Write a function that calculates the running total
- Write a function that prints a formatted report

Key Point: Each function should do exactly one job and pass data via parameters/return values

Extension Activity: Add a reporting function that groups totals by category



ACTIVITY



Peer Review Activity

The Collaborative Task:

- **Swap Code:** Trade your completed **Expense Calculator System** script with a peer
- **Trace Data Flow:** Trace how variables pass into parameters and out through return statements
- **Formulate Feedback:** Identify structural strengths and note concrete architectural improvements

Expected Outcome:

- Complete a structured evaluation matrix for your review partner
- Document actionable refactoring steps to improve code modularity

What to Look For in Your Peers Code FIREBRAND LYCEUM EDUCATION GROUP

Function Reusability:

- Does each function serve a single, isolated purpose? (Single Responsibility Principle)
- Are there any hardcoded global variables inside the function blocks?
- Could you copy-paste a function into a completely different project without it crashing?

Structural Clarity & Naming:

- Are function names action-oriented verbs? (e.g., calculate_totals vs totals)
- Do variable names describe exactly what data they hold? (e.g., expense_list vs x)
- Is indentation uniform and easy to read?

Alignment to Design Principles:

- Is input logic completely separated from processing and math logic?
- Does data move safely via arguments and returns rather than global state modifiers?

Code Review Feedback Template

Pillar	Target Indicator	Rating (GREEN / AMBER / RED)
Reusability	Pure functions free from global dependencies	
Clarity	Descriptive, semantic names for all blocks	
Separation	Distinct boundaries for Input, Process, and Output	



FIREBRAND

LYCEUM EDUCATION GROUP

Design Evaluation Discussion

Measuring the Value of Modular Architecture

Beyond "Working Code":

- **Functional vs. Structural:** Code that runs is only half the battle
- **The Software Lifecycle:** Most time is spent maintaining, not writing
- **Architectural Impact:** Your function design directly shapes development velocity

Evaluation Pillars:

- **Collaboration:** How easily teams divide up code tasks
- **Reliability:** How quickly bugs are isolated and destroyed
- **Growth:** How smoothly the system adapts to new requirements

Teamwork & Testing Dynamics

Decoupling Code to Accelerate Workflows

Team Collaboration:

- **Parallel Pipelines:** Developers build separate functions without file collisions
- **Clear Interfaces:** Defined arguments act as data contracts between coders
- **Reduced Friction:** Team members safely avoid modifying each other's lines

Testing and Debugging:

- **Isolate Faults:** Bugs are tracked to single functions instantly
- **Unit Testing:** Test logic blocks in absolute isolation easily
- **Fewer Side Effects:** Changing modular code rarely breaks distant components

Scalability & Future Growth

Designing Software for Rapid Scale

Future Enhancements:

- **Plug-and-Play:** Add new modules without rewriting existing infrastructure
- **Central Updates:** Fix an algorithm once to apply it universally
- **Low Coupling:** High independence preserves structural integrity during overhauls

Monolithic vs. Scalable Costs:

- **Monoliths:** Slower development speeds as the file size grows
- **Modular Code:** Stable development speeds regardless of code base scale

Stretch and Challenge

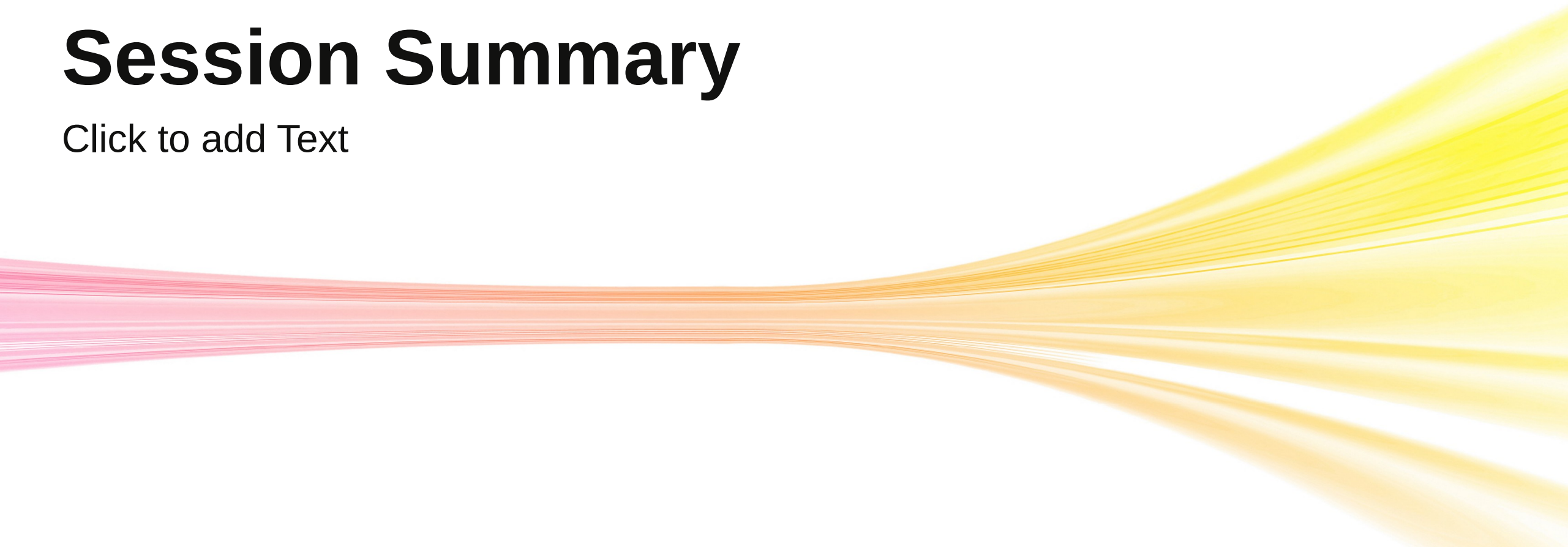
The Mission:

Upgrade your working **Expense Calculator System** into production-grade code

- **Task 1:** Explore the concept of recursion
- **Task 2:** Compare recursive and iterative approaches
- **Task 3:** Identify scenarios where each approach is appropriate

Session Summary

Click to add Text



Lesson Aim and Objectives

Lesson Aim: Functions and Code Modularity

Lesson Objectives

Apply

- Apply functions to modularise and organise code

Analyse

- Analyse code to identify duplication and inefficiencies

Evaluate

- Create reusable components aligned to design principles

Competencies and Coverage

Professional Discussion underpinned by Portfolio

Knowledge

K1, K3, K4, K5, **K7**, K8,
K10, K12

Skills

S2, S3, S5, S8, S9, S13,
S14, S15, S17

Behaviours

B1, B4, B5, B6, B7, B8,
B9

Work-based Project with Questioning

Knowledge

K2, K6, K9, K11

Skills

S1, S4, S6, **S7**, S10,
S11, S12, S16

Behaviours

B2, B3

